

The Australian National University
2600 ACT | Canberra | Australia



Australian
National
University

School of Computing

College of Systems and Societies
(CSS)

A Verified Module System for HOL4

— Honours project (S1/S2 2025)

A thesis submitted for the degree
Bachelor of Computing (Honours)

By:

Matthew Richards

Supervisor:

Dr. Michael Norrish

October 2025

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

October, Matthew Richards

Acknowledgements

I would like to thank my supervisor Dr. Michael Norrish for continuous help over the year, lengthy meetings, and many helpful ideas.

Abstract

In this thesis we propose a specification for a module system for the HOL family of theorem provers. We implement the specification for Candle, a verified ITP, and begin to prove its correctness. The module system is written as the introduction of three new inference rules afforded by the system, and the proof of correctness aims to show that these rules are conservative; that is, everything previously derivable remains derivable, and we do not allow anything new to be derived that was not previously. The final proof is not yet complete, but a large library of lemmas shows that the final result is promising, although as of now remains conjecture. Other projects for other theorem provers have presented module systems and theoretical proofs for their correctness, but to the best of the author’s knowledge this is the first to attempt their mechanised verification and implementation into a verified theorem prover. The significance of this lies in the trust that the written proofs hold over the specification. The proposed specification represents a “version 1” of the final system. What we describe is a restrictive version of an ideal module system, and cannot handle introducing polymorphic types to, for example, instantiate theorems about monoids as ones over polymorphic lists. We discuss how these limitations could be remedied, and how the current incomplete library of proofs could be extended and completed.

Keywords: formal methods, interactive theorem proving, higher-order logic, theory interpretation, prover soundness

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Preliminaries	2
1.3	Contributions	3
1.4	Thesis structure	3
2	Background	5
2.1	Notation	5
2.2	Theorem provers and their kernels	6
2.2.1	Substitutions and free variables	6
2.2.2	Proving and the <code>thm</code> type	7
2.3	Syntax and semantics in logic	7
2.4	Code extraction from HOL4	8
2.5	Candle	8
2.5.1	An approach to ITP development	8
3	Extending the Candle Kernel	9
3.1	Design goals for the usability of the system	9
3.2	The Candle backend	10
3.2.1	Project structure	10
3.2.2	Types and terms	10
3.2.3	Representing a theory	11
3.2.4	Inference rules	11
3.3	Type substitutions	11
3.4	Term substitutions	13
3.5	A new representation for theories	13
3.6	The necessary conditions for a substitution	14
3.7	The new inference rules	16
3.7.1	Eliminable axiom usage	16
3.7.2	Eliminable axiom discharge	16
3.7.3	Eliminable axiom instantiation	17
3.8	Worked example: monoids	17

Table of Contents

4	Proving the Soundness of the Extension	21
4.1	Proof strategies and details	21
4.1.1	Proving theorems about substitution	21
4.1.2	Proving equivalence of two systems	23
4.1.3	Proving theorems about derivability	24
4.1.4	An alternative approach to substitution	25
4.2	Proving the new kernel is a conservative	27
4.2.1	Safety of type substitutions	30
4.2.2	Eliminables instantiation and type instantiation	30
4.2.3	Eliminables instantiation and variable instantiation	31
4.2.4	Instantiations over abstractions	31
4.2.5	Miscellaneous supporting theorems	33
4.3	Limitations	34
4.3.1	The usefulness of the system	35
4.3.2	Implementing the specification	35
4.3.3	Limitations of the substitution functions	35
5	Related Work	37
6	Discussion and Future Work	39
6.1	Continuing the proof	39
6.2	Porting the system to HOL4	39
6.3	Permitting polymorphic substitutions	39
6.4	A direct proof of soundness	40
7	Conclusions	43
	Bibliography	45

Introduction

1.1 Motivation

Interactive theorem provers enforce rigour. When we define a new data structure, there are always many trivial lemmas we might have hand-waved the details of in a paper proof. In an ITP, we must show the same detailed and correct reasoning as always; this can quickly feel tedious and repetitive. It is easy to imagine that implementing some abstractions to make this process less tedious are desirable, but it is easier said than done. ITPs generally have a small, trusted, and very carefully written kernel, and these kinds of extensions have to be done with precision. After all, these systems have a reputation to uphold, and if the community believes they could have an unsound logic they would not be used.

Many mainstream ITPs have their own answers to the problem of tediousness in the form of a module system: prove a set of standard theorems once about abstract structures, then every data structure that is a class of these abstract structures can inherit their respective theorems for little extra effort.

This thesis deals with the HOL-family of ITPs that do not have this functionality; the most popular being HOL4. We describe an extension to these systems to introduce a module system feature. This specification is implemented for the verified theorem prover *Candle* (also based on HOL). To show that we are not compromising the correctness of its kernel, we begin to prove that the implementation is sound. The proof is incomplete, however, most of the work has been mechanised including a large library of related lemmas.

1.2 Preliminaries

Before we get started, it is important to be clear on what we concretely mean by *module system*. We will go through a brief example to ensure the thesis' goal remains clear throughout.

We will show how a module system can be used on an example of monoids to obtain theorems for natural numbers. We follow a well-known example taken from [Enderton \(1972\)](#):

Definition 1 (Monoid). A monoid is an algebraic structure $(M, \cdot, e)$ where:

- M is a set
- $\cdot : M \times M \rightarrow M$ is a binary operation
- $e \in M$ is the identity element

satisfying the following laws:

Law 1: Associativity

$$\forall a, b, c \in M : (a \cdot b) \cdot c = a \cdot (b \cdot c) \quad (1.1)$$

Law 2: Left Identity

$$\forall a \in M : e \cdot a = a \quad (1.2)$$

Law 3: Right Identity

$$\forall a \in M : a \cdot e = a \quad (1.3)$$

This is enough information to prove some basic results about monoids:

Theorem 1. [*Identity Uniqueness*] In any monoid $(M, \cdot, e)$, the identity element is unique.

Proof. Suppose there exist two identity elements e_1 and e_2 in M . Then by the definition of identity:

$$e_1 \cdot e_2 = e_2 \quad (\text{since } e_1 \text{ is an identity}) \quad (1.4)$$

$$e_1 \cdot e_2 = e_1 \quad (\text{since } e_2 \text{ is an identity}) \quad (1.5)$$

Therefore, $e_1 = e_2$, proving uniqueness of the identity element. $\square$

While not a particularly interesting result, the key fact is that this could now act as a proof for anything that satisfies our definition of a monoid. If we wanted to show the same theorem but for the natural numbers, we could simply show these satisfy the laws we described in Definition 1, choosing addition as the binary operator. Then, any proof about monoids can be carried over to the natural numbers.

Theorem 2. *[Identity Uniqueness of Zero for $\mathbb{N}$] Zero is a unique identity for the natural numbers.*

Proof. Firstly, we show that the natural numbers under addition form a monoid: this is direct, as the three monoid laws follow from the natural numbers’ axioms.

By Theorem 1, and that $(\mathbb{N}, +, 0)$ forms a monoid, the result is immediate. $\square$

We notice that convenient reasoning allows us to invoke the inference “we proved this for a certain class of data structure already, so showing that our concrete type fits into that class is sufficient”—in HOL, there is no such inference rule. We would have to completely reconstruct the former proof in terms of the new type, despite knowing that all properties relied on to write it would hold for the concrete type. This is crux of this thesis: we extend the HOL kernel to allow users to employ this kind of reasoning, resulting in minimised boilerplate work.

1.3 Contributions

Module systems in theorem provers are not a novel idea; however, the feature is new to the HOL family of theorem provers that we are looking at. We implement modules as first-class citizens in the logic. This differentiates our contribution from other HOL-based theorem provers that implement their modules as simple abstraction layers such as Isabelle without affecting the logic.

Furthermore, to the best of the author’s knowledge, this project is the first of this kind of module system to be attempted to be mechanically verified. Attempts to verify module systems exist for other ITPs, but they remain theoretical. The emerging field of theorem prover verification means it has not been attempted in such a comprehensive way yet. This is elaborated on in Chapter 5. Although the proof is incomplete, a library of completed lemmas about the result cause us to both believe the final result is true, and also minimises the amount of future work required to finish it. Time constraints are the main reason for its incompleteness; however, we discuss how the work could be continued in Chapter 4.3.

This work builds upon the formalisation and verification of Candle by [Abrahamsson et al. \(2022\)](#), and utilises their definitions and lemmas at various points. Despite this, all results presented (for example: theorems, lemmas, or definitions) are original contributions, and stated and proven as part of this project unless explicitly attributed otherwise.

1.4 Thesis structure

The thesis opens with a Chapter 2 on important background: here we will explain the notation used in this paper, and the necessary preliminaries to discuss theorem proving

1 Introduction

and what a verified module system entails.

Chapter 3 explains the design of the module system, and shows its implementation details. Here, definitions for the key functions will be given such that the full specification of the feature is stated and justified. The chapter closes by revisiting the motivating example of monoids and natural numbers, but presents it as a worked example of a derivation in the extended logic.

Chapter 4 shows the work done to verify the specification from Chapter 3. The chapter is split into three main sections: (a) we begin by discussing key proof strategies that we must use; (b) we then show how the correctness theorem is stated and proven for the specification in Chapter 3; then (c) we close by discussing limitations of the system.

Chapter 5 contextualises our work. Here, we survey related papers on module systems, both in other mainstream theorem provers, and also more theoretical papers on logical frameworks.

Chapter 6 discusses potential future work. We discuss the key choices made in the project, and the other options available for further research. We also revisit the limitations presented in Chapter 4 and discuss how these could be remediated to improve the system.

Finally, Chapter 7 concludes our thesis by discussing results and how the project can be continued in future work.

Background

2.1 Notation

The thesis follows the implementation and verification of a module system in the HOL4 theorem prover, hence all work is written in the prover's syntax. This section will give an overview of what this looks like, and how to interpret this when definitions and theorems are stated in later sections.

HOL4's syntax is fairly familiar. It is written similarly to higher-order logic as it can be found anywhere, using the expected Unicode symbols to represent quantifiers and the regular set of logical operators.

$$\begin{aligned} \forall x. (P(x) \Rightarrow Q(x)) \wedge \exists y. R(x, y) &\Longrightarrow \exists y. Q(y) \wedge R(x, y). \\ \neg(\exists x. P(x)) &\Leftrightarrow \forall x. \neg P(x). \end{aligned}$$

When used as a programming language, it reads like functional programming as it would be seen in the ML family. Definitions tend to exhibit ad-hoc polymorphism.

```
tm_names (Var n _) = [n]
tm_names (Const _ _) = []
tm_names (Comb t1 t2) = tm_names t1 ++ tm_names t2
tm_names (Abs v body) = tm_names v ++ tm_names body.
```

Record syntax is also prevalent in this paper.

```
record literal:    r = ⟨x := a; y := b; z := c⟩
record projection: x r = a,   y r = b,   z r = c
single-field update:  r' = r with x := a'
multi-field update:  r'' = r with ⟨x := a'; y := b'⟩
```

2 Background

Finally, operators for finite maps are also commonly referenced—generally in the context of the substitution map σ . For a finite map fm , consider a key k mapping to a value v , these are related by the total function **FLOOKUP** and its partial equivalent: the infix single-apostrophe. The domain and range of the maps are represented by the sets **FDOM** fm and **FRANGE** fm respectively. We also employ an infix $\cup$ to refer to the union of two finite maps (**FUNION**). Finally, the empty map is represented by the constant **FEMPTY**.

$$\begin{aligned}\text{FLOOKUP } \text{fm } k &= \text{SOME } v \Rightarrow \text{fm}' k = v \\ \text{FDOM } \text{fm} &= \{ k \mid \exists v. \text{FLOOKUP } \text{fm } k = \text{SOME } v \} \\ \text{FRANGE } \text{fm} &= \{ v \mid \exists k. \text{FLOOKUP } \text{fm } k = \text{SOME } v \} \\ \text{FUNION } \text{fm1 } \text{fm2} &= \text{fm1} \cup \text{fm2}\end{aligned}$$

2.2 Theorem provers and their kernels

In the following section, approaches to implementing a theorem prover will be described. The aim of this is to give context when discussing parts of the Candle source code, but also to motivate theorems discussed in Section 4—it often ends up being easier to transport a term to a different representation where it is easier to prove certain properties.

2.2.1 Substitutions and free variables

This entire thesis is arguably an exercise in substitution: to instantiate a module we simply substitute the abstract types and terms to the target concrete ones. According to [Pierce \(2002\)](#), substitution “turns out to be quite tricky, when examined in detail”; intuitively, however, they are simple. Generally, we want substitution to model function application. That is, when we substitute a term y into a function written in terms of a variable x , we simply return the function body with all occurrences of x replaced with y . Under naive implementations of this, issues arise quickly:

$$\begin{aligned}(x \ y)[x := t] &= t \ y \\ (\lambda y. x \ y)[x := t] &= \lambda y. t \ y \\ (\lambda y. x)[x := y] &= \lambda y. y\end{aligned}$$

The first two seem promising, but the final example does not make much sense. Our intuition seems to break down a bit when choosing the same name as the function uses results in a semantically distinct effect (in our example we have created the identity function). Clearly, we need to be able to distinguish between which variables were bound by an abstraction, and which appear free. We refer to the set of free variables by the function **FVs** : $\text{term} \rightarrow \text{term set}$; we could now claim that a substitution $E[x := y]$ is only allowed when the y we want to substitute into the term does not already appear in **FVs** E . However, this is quite limiting. We could go one step further and allow all substitutions,

and simply rename the bound variable in the function body when necessary to avoid the issues we saw in our previous third example. Substitution mechanisms that avoid issues in this way are called *capture-avoiding*.

2.2.2 Proving and the `thm` type

The HOL family of theorem provers we discuss are based on the LCF architecture. This architecture is described by [Gordon et al. \(1978\)](#), where they present an abstract data type `thm` to represent a proven theorem. A kernel is then written containing functions that produce instances of `thm` that represent each inference rule in the system. For example, modus ponens is a function in the kernel `mp : thm → thm → thm`, that takes two theorems P and $P \rightarrow Q$, and produces the theorem Q . The opaqueness of this type combined with no public constructor means there is no way to obtain an instance of this type without obtaining it from the kernel—that is, theorems can *only* come from the core inference rules. Hence the correctness of an LCF-based theorem prover comes down to the trust in its kernel.

In the mid-1980s, the project was forked and the logic of LCF replaced with a version of classical higher-order logic ([Slind and Norrish, 2008](#)). This version is the common ancestor of the “HOL family” which we keep referring to, and the reason they all have similar enough architectures such that we propose one specification in this thesis and believe it could be realistically implemented in the majority of them.

2.3 Syntax and semantics in logic

We have only discussed how inference rules relate to theorems so far; however, this is not enough. While it might seem sensible to accept that there are many different logics, and inference rules decide what is true, this does not work. This can be shown by example: suppose we mixed up conjunctions and disjunctions and introduced a faulty introduction rule.

$$\Gamma \vdash P \implies \Gamma \vdash P \wedge Q$$

It is clear to see that this is worse than just having created a silly logic with inference rules nobody would want to use: by setting P to true, and Q to false, we have now derived false in our system—our inference rules are inconsistent with our notion of truth, that is, they are *unsound*. Reasoning based on inference rules is known as syntax, consequences based on truth values is known as semantics ([Enderton, 1972](#)). Our conclusion here syntactically follows, but semantically does not. It is a goal for a logic to show the equivalence of these two things.

Noting that unlike the syntactic consequence shown above, $\Gamma \models P$ is the notation for semantic consequence, we can now formally define soundness (shown in Definition 2) and completeness (Definition 3).

2 Background

Definition 2 (Soundness). Everything derivable from a logic’s inference rules is true.

$$\Gamma \vdash P \implies \Gamma \models P$$

Definition 3 (Completeness). Everything true is derivable from the logic’s inference rules.

$$\Gamma \models P \implies \Gamma \vdash P$$

2.4 Code extraction from HOL4

The HOL4 theorem prover is a system for writing and checking proofs in higher-order logic (Slind and Norrish, 2008). It allows users to express functional programs and write proofs about these.

Code written in HOL4 is a candidate for code extraction to the verified functional programming language *CakeML* (Myreen and Owens, 2012). Thus, verified software can be written in HOL4, then a binary produced that is verified to match its HOL specification. This leads to verified software projects such as the verified theorem prover *Candle*, which consequently serves as both a verified specification for a theorem prover, and also an executable and practical environment for theorem proving.

2.5 Candle

Candle is a verified interactive theorem prover based on HOL (Abrahamsson et al., 2022). The project specifies the entire system in HOL4 and proves a soundness theorem of its kernel. This provides us with a rich library of proofs about theorem prover correctness, ranging from reasoning about its inference rules, to the type safety of its accepted programs.

While interesting on its own, this provides an environment in which ideas for new theorem prover features can be developed, and their properties verified using Candle’s wide library of theorems in all areas of the system. It is here, then, that the project described in this thesis will be developed, with its correctness proven over Candle’s kernel.

2.5.1 An approach to ITP development

In this project, Candle is used as a development platform to specify and verify an extension for HOL ITPs. It is proposed that in future work, this specification could be lifted out onto other similar provers. This methodology has been used before in a project to add novel tactics for HOL4 (Abrahamsson et al., 2025): after verifying that the implementation is correct for Candle, the code is manually ported over to HOL4’s ML source code based on inspection of the HOL4 definitions. The success of this project motivates ours.

Extending the Candle Kernel

In this chapter we will explain the implementation of the module system for Candle, and how it fits into the existing sources. Ultimately, the goal of the section is to justify and explain the implementation. Discussion on the verification of the system is not mentioned until Chapter 4.

We begin with a high-level explanation of the design goals to achieve a usable system, before moving onto the technical aspects of the implementation.

3.1 Design goals for the usability of the system

Before any implementation details of the system can be discussed, it is clear that some preliminary discussion on design goals is needed. We have described what a module system entails for the purposes of this project; this subsection will introduce what that should end up looking like—in high level—to motivate the following content. At the end of this section, we revisit the monoid example from the introduction to show how our extended system would handle it, and to show how it fits into our design goals. We propose as these design goals that the system should be able to do the following:

- **Support reusable abstraction:** The system should allow users to define parameterised collections of theorems and later instantiate them on concrete types and constants. This enables proof reuse over common algebraic structures, reducing repeated proofs.
- **Minimal work to apply a module:** The purpose of the system is to make the user's life easier; once a module has been written, it should be as simple as possible to instantiate it and start using its transported theorems on the concrete type.
- **Integrate into existing theories:** HOL4 is already in heavy use. The system should be able to be used to enrich existing theories.

3.2 The Candle backend

We now take a break from discussing our original work to explain the key parts of the Candle sources that we must modify to implement our system.

3.2.1 Project structure

A subset of the project structure that we are interested in is shown in Figure 3.1. This shows the split between syntax and semantics that we described in Chapter 2. As our

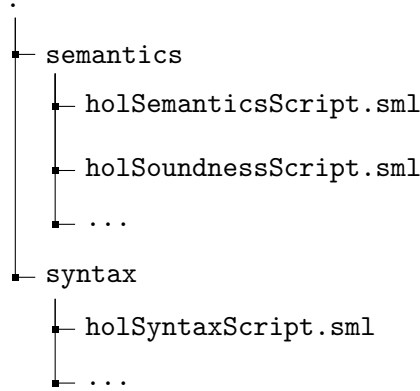


Figure 3.1: A relevant subset of the Candle sources

goal is to introduce new proof steps that instantiate abstract theorems, the majority of our work must be done in `holSyntaxScript.sml`; new inference rules are syntactic changes. We discuss semantics only as part of potential future work in Chapter 6.

3.2.2 Types and terms

The constructors for the term representations in Candle are shown in Definition 4. Notably, variables are products of names and types. Constants represent primitive parts of the language; for example, `true`, `false`, `+`, and `1` are all constants. `Comb` is our function application construct, i.e., `f x` is represented at this level as `Comb f x`. Finally, `Abs` represents lambda abstractions—the first argument is the bound variable, and the second is the body.

Definition 4. The constructor for terms in Candle.

```

term ::= Var mlstring type
      | Const mlstring type
      | Comb term term
      | Abs term term
  
```

The constructors for types are shown below in Definition 5. We can see the separation between type variables and type applications. Type variables represent polymorphic types, for example, the type α as one would see it in other languages such as Haskell. Type applications represent constructors for types; the string representing the used constructor, for example, the polymorphic $\alpha \rightarrow \alpha$ is internally represented by `Tyapp "fun" (Tyvar "a") (Tyvar "a")`. A nullary type (one with no constructor arguments) such as `bool` is `Tyapp "bool" []`.

Definition 5. The constructor for types in Candle.

```
type ::= Tyvar mlstring
      | Tyapp mlstring type list
```

3.2.3 Representing a theory

Previously, we have denoted syntactic entailment by equations such as:

$$\Gamma \vdash P$$

In Candle, this relation would have type `thm`, $\Gamma : \text{thy} \times \text{term list}$, and $P : \text{term}$. We separate Γ into a pair of `thy` and a `term list`, the former containing the signature (the registered constants and types) and axioms present in a theory; and the latter being the hypotheses under which the conclusion P is derived.

3.2.4 Inference rules

Candle encodes its syntactic relations by an inductive relation called `proves`, notated by `thy | - c`, and with the type `thm`; this enforces the fact that the only way to obtain an instance of the `thm` type is through inference rules, as we described in Chapter 2.

Figure 3.2 shows the full list of available inference rules as they are written in the Candle kernel. We can analyse common traits of these rules to justify our decisions in the following sections. We make the following notes:

- The theory must always be “ok”, i.e., all the defined constants must have registered types, the axioms must be valid boolean propositions, and booleans and equality must be defined.
- Hypotheses must be `term_ok`, mainly requiring them to be well-typed and to be made up of registered types and terms from the signature.

With these preliminaries out of the way, we now end this detour into Candle’s backend and return to our original contributions.

3.3 Type substitutions

In our module system, we want the ability to instantiate abstract types. For example, theorems that reason about monoids may want to substitute the monoid type for natural

3 Extending the Candle Kernel

numbers to obtain a concrete theorem. Two functions are written to manage different parts of these type substitutions: (a) `ty_esubst`, which substitutes types in a type, and (b) `esubst_ty0`, which substitutes types in a term (using the former function as an auxiliary).

We can firstly look at `ty_esubst` in Definition 6. We have chosen to only substitute types that are both monomorphic (no type variables) and nullary (no type constructors). While this will be justified later in the thesis, the overall reason is simplicity of both the implementation and the verification.

Definition 6. The code for the term substitution function `ty_esubst`.

$$\begin{aligned}
 \text{ty_esubst } _ (\text{Tyvar } n) &= \text{Tyvar } n \\
 \text{ty_esubst } \sigma (\text{Tyapp } n \ []) &= \begin{cases} \text{Tyapp } n \ [] & \text{if } \text{FLOOKUP } (\text{FST } \sigma) \ n = \text{NONE} \\ \text{ty} & \text{if } \text{FLOOKUP } (\text{FST } \sigma) \ n = \text{SOME } \text{ty} \end{cases} \\
 \text{ty_esubst } \sigma (\text{Tyapp } n \ \text{ts}) &= \text{Tyapp } n \ (\text{MAP } (\text{ty_esubst } \sigma) \ \text{ts})
 \end{aligned}$$

This function is used to write the *type substitution in a term* function `esubst_ty0`. It is written with an environment that it adds to when traversing abstractions, to keep track of which variables are bound and which are free. This is important so it knows which variables are safe to rename to avoid capture.

Shown in Figure 3.3, the code for type substitutions seems complex, but is based on the simple idea of avoiding capture as we have previously described it. The only difference here is that variables are now names and types. Two variables are allowed to have the same name as long as they have different types, so clearly type substitution can cause capture. From this, we give the main ideas of the function as follows:

- When substituting the type of a `Var`, we check if the target substitution is already paired with a different variable. If it is, this transformation would cause the capturing of a different variable, so we error. Otherwise, we return the substituted variable.
- In the `Const` case, we just change the type.
- In the `Comb` case, we simply recursively substitute both sides.
- In the `Abs` case, we try the straight-forward substitution—if this errors then we have to rename the bound variable and try again.

We also note that the function has an *avds* parameter. This is a list of strings that represents the names the capture-avoiding substitution should avoid when renaming bound variables. This is useful to give the function extra context of which variables have already been bound; for example, to instruct the function to avoid re-using variable names that appear in the hypotheses of a sequent.

3.4 Term substitutions

Term substitutions are far simpler. The motivation for term substitutions in the module system is simply to replace abstract constants with concrete ones; hence, traversing the term normally and looking up each `Const` in the substitution map is enough. The code is shown below in Figure 7:

Definition 7. The code for the term substitution function `esubst_tm`.

$$\begin{aligned}
 \text{esubst_tm } \sigma (\text{Var } n \text{ ty}) &= \text{Var } n \text{ ty} \\
 \text{esubst_tm } \sigma (\text{Abs } v \ t) &= \text{Abs } v \ (\text{esubst_tm } \sigma \ t) \\
 \text{esubst_tm } \sigma (\text{Comb } t_1 \ t_2) &= \text{Comb } (\text{esubst_tm } \sigma \ t_1) (\text{esubst_tm } \sigma \ t_2) \\
 \text{esubst_tm } \sigma (\text{Const } n \text{ ty}) &= \begin{cases} \text{Const } n \text{ ty} & \text{if FLOOKUP (SND } \sigma) \ n = \text{NONE} \\ \text{tm} & \text{if FLOOKUP (SND } \sigma) \ n = \text{SOME tm} \end{cases}
 \end{aligned}$$

Finally, we define `esubst` as the composition of term and type substitution, where `esubst_ty` simply calls `esubst_ty0` with an empty environment.

Definition 8. The code for the substitution function `esubst`.

$$\text{esubst } \sigma \text{ avds } tm = \text{esubst_tm } \sigma (\text{esubst_ty } \sigma \text{ avds } tm)$$

3.5 A new representation for theories

The substitution functions work as described; however, they are currently too flexible. The ability to substitute any types, and any constants, is very strong: a user could remove primitive types, or create inconsistent substitutions that result in a term that is not well-typed. It should be clear that for a module system like ours, we only intend to allow users to substitute out types and constants deemed *abstract*—though from hereon we refer to them as *eliminable*. Thus, we have extended the `thy` type to include extra fields to separate the eliminable types and terms from the concrete ones. We will only perform a substitution on a term if the target type or term is in the eliminable component of the theory. Shown in Definition 9, the `thy` type is a tuple of a signature (registered terms and types), and a set of axioms. The new extended version now has almost double the fields (given an eliminable and concrete version of each component), and thus it has been rewritten as a record with named fields for clarity. This is depicted in Definition 10.

Definition 9. Candle’s representation of a theory, as a tuple of a signature and an axiom set.

$$\begin{aligned}
 \text{tysig} &= \text{mlstring} \mapsto \text{num} \\
 \text{tmsig} &= \text{mlstring} \mapsto \text{type} \\
 \text{sig} &= \text{tysig} \times \text{tmsig} \\
 \text{thy} &= \text{sig} \times \text{term set}
 \end{aligned}$$

Definition 10. The extended theory record definition.

$$\begin{aligned} \text{ethy} = \langle & | \text{tms} : \text{mlstring} \mapsto \text{type}; \\ & \text{tys} : \text{mlstring} \mapsto \text{num}; \\ & \text{axs} : \text{term set}; \\ & \text{etms} : \text{mlstring} \mapsto \text{type}; \\ & \text{etys} : \text{mlstring} \mapsto \text{num}; \\ & \text{eaxs} : \text{term set} \rangle \end{aligned}$$

3.6 The necessary conditions for a substitution

The notion of distinct eliminable types and terms begin to guard what substitutions are considered valid, but are still not enough on their own. This can be shown by simple examples that if performed naively, break our intuition for what we believe a module system should be allowed to do. We have defined a function `esubsts_ok` to restrict which substitutions are allowed in the system. The function takes the signature of a theory `sig`, as well as the substitution map σ . Its definition is given by Definition 11.

Definition 11. `[esubsts_ok]`

$$\begin{aligned} \text{esubsts_ok}(\text{sig}, (\sigma, \theta)) \Leftrightarrow \\ \text{strlit}(\text{"="}) \notin \text{FDOM}(\theta) \end{aligned} \tag{3.1}$$

$$\wedge \text{strlit}(\text{"bool"}) \notin \text{FDOM}(\sigma) \tag{3.2}$$

$$\begin{aligned} \wedge (\forall \text{tmnm.tnm} \in \text{FDOM}(\theta) \Rightarrow \\ \exists \text{ty.FLOOKUP}(\text{tms}(\text{sig}), \text{tmnm}) = \text{SOME}(\text{ty}) \wedge \\ \text{is_monomorphic}(\text{ty}) \wedge \\ \text{typeof}(\theta(\text{tmnm})) = \text{ty_subst}((\sigma, \theta), \text{ty})) \end{aligned} \tag{3.3}$$

$$\begin{aligned} \wedge (\forall \text{ty.ty} \in \text{FRANGE}(\sigma) \Rightarrow \\ \text{type_ok}(\text{tysof}(\text{sig}), \text{ty}) \wedge \\ \text{is_monomorphic}(\text{ty})) \end{aligned} \tag{3.4}$$

$$\begin{aligned} \wedge (\forall \text{tm.tm} \in \text{FRANGE}(\theta) \Rightarrow \\ \text{term_ok}(\text{subst_sig}((\sigma, \theta), \text{sig}), \text{tm}) \wedge \\ \exists n, \text{ty.ty} = \text{Const}(n, \text{ty})) \end{aligned} \tag{3.5}$$

$$\wedge \text{DISJOINT}(\text{FDOM}(\sigma), \text{BIGUNION}(\text{IMAGE}(\text{nullary_ops_of}, \text{FRANGE}(\sigma)))) \tag{3.6}$$

In words, the conjuncts of this definition can be understood as follows:

- (3.1) The term substitution map does not try to substitute out the equals constant;
- (3.2) The type substitution map does not try to substitute out the boolean type.

These first two conditions are important as these make up the HOL “standard signature”. The inference rules themselves rely on booleans and equality being defined; the system would break if these were substituted away.

- **(3.3) All terms that are to be substituted out have a registered *monomorphic* type in the signature, that respects the mapped type substitutions.**

There are two components in play here. We will look at the importance of “respecting type substitution” first.

Consider the term “ $\lambda(x : \text{bool}). \neg x$ ”, where $(\neg)$ is a constant of type $\text{bool} \rightarrow \text{bool}$. Under the type substitution $\text{bool} \mapsto \text{int}$ ¹ and the term substitution $\neg \mapsto \text{SUC}$, we apply the type substitutions first, to obtain “ $\lambda(x : \text{int}). \neg x$ ”. It is now obvious that the type of the term we replace $\neg$ with must have the substituted type, that being $\text{int} \rightarrow \text{int}$.

Now, we will look at an example to show the terms being replaced must have monomorphic types—this seems less intuitively clear, although the reason is quite straight-forward. Consider the term “ $\lambda(x : \text{int}). \text{id } x$ ”, where $\text{id} : \alpha \rightarrow \alpha$. In the term, HOL instantiates the type variables by replacing α with int , but id retains its polymorphic type in the signature. Thus, the respecting type substitution clause does not hold as it compares the type in the term with the type in the signature, and they now do not match.

- **(3.4) Every type that is substituted in must be valid with respect to the signature, and must be monomorphic;**

The first clause here is hopefully already clear (everything that we want to insert into a term needs to have a valid type). However, its type must also be monomorphic. This is a requirement as we do not want to be introducing *new* type variables during a substitution, as this makes our proof obligations far more complex.

- **(3.5) Every term that is substituted in must be valid with respect to the *substituted* signature and must be a constant.**

The first component is again fairly straight-forward: in the new signature that contains the substituted types and terms, we need to be sure all of our terms we have substituted in are valid. These terms must also be a constant as our implementation of `esubst_tm` only substitutes out constants—for simplicity we only replace them with other constants.

- **(3.6) The domain of the type substitution and the nullary operators of its range must be disjoint.**

This final conjunct has a relatively simple reason: we want substitution to be idempotent. This is easy if we never introduce something that we also want to substitute out. Given we only substitute nullary type operators, these are the ones we look for.

¹This example is technically disallowed as we have established we cannot substitute out the boolean type... but it will do for this example.

3.7 The new inference rules

We have implemented three new inference rules to achieve the functionality of a module system. Together, they allow users to reason with eliminable terms/types/axioms present in the theory, then they are discharged from the signature once they are instantiated. This makes it easy to observe when a theory is “infected” with eliminable parts, compared to when it is “pure” (when there are no eliminable parts present in the theory, or they have all been discharged).

3.7.1 Elimidable axiom usage

The first rule `elim_axioms` allows the user to reason with eliminable axioms. An eliminable axiom defined in the theory can be proven, but after use it is added into the `es` set. This keeps track of all the used eliminable axioms, so the system knows which must be discharged against the concrete types/terms when instantiating the theorem.

For example, when proving the uniqueness of 0 as an identity element using the uniqueness of the monoid identity, the first step is to invoke the monoid version; now, even if we instantiate this onto the natural numbers, the `es` set has the eliminable axiom in it. This is what notifies the system that the user is yet to demonstrate how the monoid axioms fit on the natural numbers.

Rule 1. *The `elim_axioms` inference rule*

$$\frac{\text{theory_ok}' \text{ thy} \quad c \in \text{thy.eaxs}}{(\text{thy}, \{c\}, []) \vdash' c}$$

3.7.2 Elimidable axiom discharge

`elim_discharge` is the previously alluded to rule to discharge axioms from the `es` set.

Rule 2. *The `elim_discharge` inference rule*

$$\frac{(\text{thy}, es_1, []) \vdash' c_1 \quad (\text{thy}, es_2, h_2) \vdash' c_2}{(\text{thy}, ((es_2 \setminus \{c_1\}) \cup es_1), h_2) \vdash' c_2}$$

While it is not necessarily so clear how this rule applies to the described scenario, we can suppose the left side of the antecedent depicts deriving an eliminable axiom used on the right side only using standard axioms. For example, if we were in the theory of natural numbers, the monoid eliminable axioms can be derived from the axioms of the natural numbers—left/right identity and associativity are standard in $\mathbb{N}$. This rule ultimately states that once we show our eliminable axioms are deriveable in our theory, we can remove the dependency on these from any other sequent. In case the rule is still unclear, there is a worked example in Section 3.8.

3.7.3 Elimidable axiom instantiation

The third and final inference rule `elim_inst` transports a valid sequent through a well-typed interpretation σ (of types and terms), yielding a concrete instance.

$$\frac{\begin{array}{c} (thy, es, h) \vdash' c \\ \text{theory_ok}'\ thy \\ \text{esubsts_ok}(thy.ctys, thy.ctms)\ \sigma \\ \text{esubsts_total}\ thy\ \sigma \end{array}}{(thy_\sigma, es_\sigma, h_\sigma) \vdash' \text{esubst}\ \sigma\ \text{avds}\ c}$$

This rule underpins the abstract-concrete transportation central to the module system: given any valid sequent, as long as the substitutions are valid, the substituted sequent is also valid. This applies to the theory itself (elimidable terms must have their registered types substituted), the `es` set (we discharge substituted elimidable axioms, not the abstract one), the hypotheses, and the conclusion.

To follow the monoid example: this allows us to interpret our monoid sequents in the theory of the natural numbers, and force the user to discharge the monoid elimidable axioms in terms of natural numbers. Of course, it would almost never be possible to discharge the elimidable axioms in terms of elimidable types/terms without using more elimidable axioms.

3.8 Worked example: monoids

We finally make it to a point where we can go through an example. We will reflect on the motivating example we provided in Chapter 1 and restate it in terms of the extended logic we have constructed. We begin by defining the monoid theory thy_M and the natural number theory $thy_{\mathbb{N}}$ with only a subset of the natural number axioms for simplicity.

```
thy_M = ⟨ | tms := FEMPTY;
           tys := FEMPTY;
           etms := {e : monoid; (·) : monoid → monoid → monoid};
           etys := {monoid};
           axs := ∅;
           eaxs := {∀a, b, c. (a · b) · c = a · (b · c); ∀a. e · a = a; ∀a. a · e = a} | ⟩
```

3 Extending the Candle Kernel

$$\begin{aligned}
thy_{\mathbb{N}} = \langle & | \text{tms} := \{0 : \mathbb{N}; \text{Suc} : \mathbb{N} \rightarrow \mathbb{N}; (+) : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}\}; \\
& \text{tys} := \{\mathbb{N}\}; \\
& \text{etms} := \text{FEMPTY}; \\
& \text{etys} := \text{FEMPTY}; \\
& \text{axs} := \{\forall a, b, c. (a + b) + c = a + (b + c); \forall a. 0 + a = a; \forall a. a + 0 = a\}; \\
& \text{eaxs} := \emptyset \rangle
\end{aligned}$$

First we prove the uniqueness of the identity of monoids to obtain the sequent (with the empty list for its hypothesis).

$$(thy_M, used_eaxes, []) \vdash \forall k. (\forall a. k \cdot a = a) \implies k = e$$

where $used_eaxes = \{\forall a, b, c. (a \cdot b) \cdot c = a \cdot (b \cdot c); \forall a. e \cdot a = a; \forall a. a \cdot e = a\}$ The sequent is derived by standard reasoning, and appealing to `elim_axioms` to use the eliminable monoid axioms when required; noting that when these are used they are added to the $used_eaxes$ set.

We want to now transport this sequent into the natural numbers theory, hence we take the substitutions σ that map the eliminable type `monoid` to $\mathbb{N}$, and the eliminable terms $(\cdot)$ and e to $(+)$ and 0 respectively. We use σ and the `elim_inst` inference rule to obtain the transported sequent.

$$(thy_{\mathbb{N}}, used_eaxes_{\mathbb{N}}, []) \vdash \forall k. (\forall a. k + a = a) \implies k = 0$$

where $used_eaxes_{\mathbb{N}} = \{\forall a, b, c. (a + b) + c = a + (b + c); \forall a. 0 + a = a; \forall a. a + 0 = a\}$

Before we can freely use this theorem, we must discharge the set of used eliminable axioms. Each of these corresponds to an existing axiom in $thy_{\mathbb{N}}$. Recall that `elim_discharge` takes a witness sequent that shows an eliminable axiom can be derived in a theory with relying on the axiom, and a target sequent from which we want to discharge its $used_eaxes$. Hence we can invoke this rule three times to obtain the final sequent:

$$(thy_{\mathbb{N}}, \emptyset, []) \vdash \forall k. (\forall a. k + a = a) \implies k = 0$$

$$\begin{array}{c}
\frac{\neg \text{EXISTS } (\text{VFREE_IN}(\text{Var } x \text{ ty})) \ h \quad \text{type_ok } (\text{tysof } \text{thy}) \ \text{ty} \quad (\text{thy}, h) \vdash l \equiv r}{(\text{thy}, h) \vdash \text{Abs } (\text{Var } x \text{ ty}) \ l \equiv \text{Abs } (\text{Var } x \text{ ty}) \ r} \text{ABS} \\
\\
\frac{\text{theory_ok } \text{thy} \quad p \text{ has_type Bool} \quad \text{term_ok } (\text{sigof } \text{thy}) \ p}{(\text{thy}, [p]) \vdash p} \text{ASSUME} \\
\\
\frac{\text{theory_ok } \text{thy} \quad \text{type_ok } (\text{tysof } \text{thy}) \ \text{ty} \quad \text{term_ok } (\text{sigof } \text{thy}) \ t}{(\text{thy}, []) \vdash \text{Comb } (\text{Abs } (\text{Var } x \text{ ty}) \ t) \ (\text{Var } x \text{ ty}) \equiv t} \text{BETA} \\
\\
\frac{(\text{thy}, h_1) \vdash p \equiv q \quad (\text{thy}, h_2) \vdash p' \quad \text{ACONV } p \ p'}{(\text{thy}, \text{term_union } h_1 \ h_2) \vdash q} \text{EQMP} \\
\\
\frac{(\text{thy}, h_1) \vdash c_1 \quad (\text{thy}, h_2) \vdash c_2 \quad h = \text{term_union } (\text{term_remove } c_2 \ h_1) \ (\text{term_remove } c_1 \ h_2)}{(\text{thy}, h) \vdash c_1 \equiv c_2} \text{DEDUCTANTISYM} \\
\\
\frac{\forall s \ s'. \text{MEM } (s', s) \ \text{ilist} \Rightarrow \exists x \ \text{ty}. s = \text{Var } x \ \text{ty} \wedge s' \text{ has_type ty} \wedge \text{term_ok } (\text{sigof } \text{thy}) \ s' \quad (\text{thy}, h) \vdash c}{(\text{thy}, \text{term_image } (\text{VSUBST } \text{ilist}) \ h) \vdash \text{VSUBST } \text{ilist } c} \text{INST} \\
\\
\frac{\text{EVERY } (\text{type_ok } (\text{tysof } \text{thy})) \ (\text{MAP FST tyin}) \quad (\text{thy}, h) \vdash c}{(\text{thy}, \text{term_image } (\text{INST tyin}) \ h) \vdash \text{INST tyin } c} \text{INSTTYPE} \\
\\
\frac{(\text{thy}, h_1) \vdash l_1 \equiv r_1 \quad (\text{thy}, h_2) \vdash l_2 \equiv r_2 \quad \text{welltyped } (\text{Comb } l_1 \ l_2)}{(\text{thy}, \text{term_union } h_1 \ h_2) \vdash \text{Comb } l_1 \ l_2 \equiv \text{Comb } r_1 \ r_2} \text{MKCOMB} \\
\\
\frac{\text{theory_ok } \text{thy} \quad \text{term_ok } (\text{sigof } \text{thy}) \ t}{(\text{thy}, []) \vdash t \equiv t} \text{REFL} \quad \frac{\text{theory_ok } \text{thy} \quad c \in \text{axsof } \text{thy}}{(\text{thy}, []) \vdash c} \text{AXIOM}
\end{array}$$

Figure 3.2: Inference rules for the proves relation.

```

esubst_ty0 env σ avds (Var n ty) = let tm := Var n ty; tm' := Var n (ty_esubst σ ty)
                                   in if REV ASSOCD tm' env tm = tm then return tm'
                                   else error tm

esubst_ty0 env σ avds (Const n ty) = return (Const n (ty_esubst σ ty))

esubst_ty0 env σ avds (Comb t1 t2) = do t'1 ← esubst_ty0 env σ avds t1;
                                           t'2 ← esubst_ty0 env σ avds t2;
                                           return (Comb t'1 t'2) od

esubst_ty0 env σ avds (Abs v body) = let (n, ty) := dest_var v;
                                       ty' := ty_esubst σ ty; v' := Var n ty'
                                       in try do body' ← esubst_ty0 ((v, v') :: env) σ avds body;
                                       return (Abs v' body') od
                                       (λbad_v. if bad_v ≠ v' then error bad_v else
                                       do body1 ← esubst_ty0 [] σ avds body;
                                       n' := NVARIANT n avds body1;
                                       body' := VSUBST [(Var n' ty, Var n ty)] body;
                                       env'' := (Var n' ty, Var n' ty') :: env;
                                       body'' ← esubst_ty0 env'' σ avds body';
                                       return (Abs (Var n' ty') body'') od)

```

Figure 3.3: Definition of `esubst_ty0`

Proving the Soundness of the Extension

We begin to prove the extension described in Chapter 3 to be conservative. In this context, this describes the properties that (a) everything provable before is still provable, and conversely, (b) no new theorems can now be proven that could not be proven in the old system. This immediately entails soundness, as there is an existing proof in the Candle sources that everything currently provable is true.

In what follows, proofs for these two main theorems will be presented. The first is fairly straight-forward; the second makes up the bulk of the effort of the thesis, and remains incomplete. Many auxiliary lemmas are required to prove this result—we have proven a number of these. Despite the final theorem not being completely proven, all presented lemmas and theorems in this chapter are fully proven in HOL4 unless explicitly stated otherwise. The chapter begins with a section on proof strategies, describing how proofs on different types of goals are generally approached. Following this, we present the main two theorems and discuss their respective journeys.

4.1 Proof strategies and details

4.1.1 Proving theorems about substitution

There are three main substitution functions we have proven the key results for: substituting types in a term, substituting constants in a term, and substituting types in a type.

Theorems about type substitutions in terms are stated using `esubst_ty0`, and are proven using recursive induction over its cases—HOL4 automatically generates the induction principle for inductively defined functions. The first issue that arises when using this induction principle surrounds how `esubst_ty0` refers to an association list in its parameters that keep track of bound variables at a given point when traversing a term. Despite the

4 Proving the Soundness of the Extension

case of the empty environment being ideal for a practical theorem, being too direct about what we want to prove creates an induction hypotheses that is too strong. To see this, consider a theorem that intends to prove `esubst_ty0` preserves the size of a term (taking `sizeof tm` to have a sensible recursive definition).

$$\forall \sigma \text{ avds } tm \text{ } tm'. \text{ esubst_ty0 } \sigma \text{ avds } tm = \text{return } tm' \implies \text{sizeof } tm' = \text{sizeof } tm$$

Inducting on the cases of `esubst_ty0`, the `Var` and `Const` cases follow almost immediately: the former must stay the same as the variable binding environment is empty, and the latter simply has its type replaced. The `Comb` and `Abs` cases require the inductive hypothesis. `Comb` is not a hard proof: the two sub-terms match against their inductive hypotheses, and the `sizeof` call sums them back together. Finally, we can proceed to the `Abs` case, where we traverse into the abstraction body and add the binder to the environment. However, with the environment now non-empty, the inductive hypothesis does not apply as it has a fixed empty environment—we are stuck. This becomes the standard problem in all of the related theorems: the goal must be weakened to tolerate any environment. Consequently, most theorems have a quantified `env`, and include the standard “realistic environment” requirement shown below:

$$\forall k, v. \text{MEM}(v, k) \text{ env} \implies \exists n \tau. k = \text{Var } n \text{ (ty_esubst } \sigma \tau) \wedge v = \text{Var } n \tau$$

Term substitutions are the most straight-forward: the function only affects constants, and otherwise continues to traverse the term. Hence the standard induction principle for terms is sufficient, shown in Figure 4.1.

$$\frac{\begin{array}{l} \forall n \text{ ty}. P(\text{Var } n \text{ ty}) \\ \forall n \text{ ty}. P(\text{Const } n \text{ ty}) \\ \forall tm_1 \text{ } tm_2. P(tm_1) \wedge P(tm_2) \implies P(\text{Comb } tm_1 \text{ } tm_2) \\ \forall tm_1 \text{ } tm_2. P(tm_1) \wedge P(tm_2) \implies P(\text{Abs } tm_1 \text{ } tm_2) \end{array}}{\forall tm. P(tm)}$$

Figure 4.1: The induction hypothesis for the term representation in Candle. It is automatically generated from its recursive definition by HOL4.

Finally, theorems about type substitutions in types use `ty_esubst`, which is not context-carrying in the same way that `esubst_ty0` caused problems by varying its environment. Proofs about these are generally by induction over the type. However, the main interesting property of this is that the generated induction principle by HOL4 is too weak. As shown in Figure 4.2, the proposition holds over the entire list, instead of every element of the list individually—this makes it practically hard to do anything with when the elements are being changed. For this reason, [Abrahamsson et al. \(2022\)](#) present a different induction hypothesis for lists, shown in Figure 4.3 to directly address this.

Effectively every proof we present in this section will be some kind of induction. Combined with the induction hypotheses presented above, we present a general proof strategy.

$$\begin{array}{c}
\forall n. P_0(\text{Tyvar } n) \\
\forall lst. P_1(lst) \Rightarrow \forall n. P_0(\text{Tyapp } n \text{ } lst) \\
P_1(\Box) \\
\hline
\forall ty \text{ } lst. P_0(ty) \wedge P_1(lst) \Rightarrow P_1(\text{ty}::lst) \\
(\forall ty. P_0(ty)) \wedge (\forall lst. P_1(lst))
\end{array}$$

Figure 4.2: The induction hypothesis for the Candle representation of types, automatically generated by HOL4.

$$\begin{array}{c}
\forall n. P(\text{Tyvar } n) \\
\forall lst. \text{EVERY } P \text{ } lst \Rightarrow \forall n. P(\text{Tyapp } n \text{ } lst) \\
\hline
\forall ty. P(ty)
\end{array}$$

Figure 4.3: The hand-written induction hypothesis for the Candle representation of types, showing a proposition over each element of the type constructor.

4.1.2 Proving equivalence of two systems

After introducing the extension in Chapter 3, we have essentially created a forked logic from HOL. Our task now is to prove it is “equivalent” to the original system. By this, we refer to the relation between two systems such that the set of derivable sentences are the same.

We could consider natural deduction and the sequent calculi for a simple example. These systems are structured differently, but can express and prove the same set of theorems. Troelstra (1996) shows that this is done by independently showing both directions: for every derivation from system A, it must be able to be represented in system B; then every derivation in system B must also have a representation in system A. Each direction is proved by induction over the relation, and each rule is examined case-by-case to show an equivalent step in the other system. On the left, we have a derivation for conjunctions in the sequent calculus; on the right, we show that the same conclusion can be reached in natural deduction.

$$\begin{array}{ccc}
\frac{\Gamma \Rightarrow A \quad \Gamma \Rightarrow B}{\Gamma \Rightarrow A \wedge B} & \iff & \frac{A \quad B}{A \wedge B}
\end{array}$$

After doing this for every case, and then in the other direction, we prove that everything derivable in one system can have an equivalent proof in the other, and vice versa.

Our task is similar, and our proof strategy reflects this. We intend to prove that everything derivable in the original Candle kernel can still be derived in the extended kernel,

and everything derivable in the extended kernel could have been derived without the extension.

4.1.3 Proving theorems about derivability

Notation. There are two distinct logics discussed in this thesis, the “old” logic, and the “new” logic, that is, classical higher-order logic as it is represented in HOL4/Candle, and the version with the three new inference rules added. Sequents in the former are notated $\text{thy} \vdash c$, while sequents in the latter depict a primed turnstile instead: $\text{thy} \vdash' c$. We aim to prove that for all theories and conclusions, these relations are equivalent.

Theorems stated with either turnstile are proved by induction over the generated induction principle based on the relation (that is, very informally, showing that when the sub-expressions are derivable, so is the larger expression that the inference rule yields). Here, the base cases are the inference rules that give the user a theorem without having anything previously proven, for example, the assumption inference rule states that “given p is an assumption in a theory, p can be derived in that theory”.

A complete induction approach. However, there is a case where this induction principle does not afford us the results we want. When working with functions over terms, it is often sensible to perform complete induction over the size of the term: this gives us a weaker induction hypothesis we can use on, for example, substituted terms rather than the base term (as these have the same size). A similar induction principle is desirable for proofs, but requires some measure of the size of a proof. Furthermore, we note that the proof of equivalence between natural deduction and the sequent calculus from Troelstra (1996) was also a complete induction over the derivation size—that is, this idea is not novel, and is in fact expected in this nature of proof.

We introduce the `nproves` relation denoted by the infix operator $(\text{thy}, h) \vdash_n c$ ¹. Here, the n represents the number of inference rules it took to derive a conclusion. This gives us the ability to perform strong induction over the number of steps it takes to derive theorems, allowing us to effectively perform strong induction over *proof size*. Now, once it is shown that the `nproves` relation is equivalent to `proves` (or at least, that there is some number of steps that makes them equivalent), difficult theorems needing strong induction can be transported from `proves` to `nproves` to perform the induction, then transported back once the result is shown. We hypothesised that this will make complex proofs more feasible.

Interestingly, this approach was not as useful as expected. We found that despite its success in literature, this method did not seem to fit our proofs. As we are reasoning about capture-avoiding substitution, many of our lemmas are only claimed up to alpha equivalence. We found that to prove an alpha-equivalent term, it took one extra “derivation step”. Therefore, when our complete induction hypothesis would allow us

¹This is represented in our HOL4 proofs as $(\text{thy}, h, n) \vdash n - c$ (where the n is a part of the infix symbol), however, for nicer reading we will pretend the value of n appears as a subscript of the turnstile.

to instantiate sequents with a derivation length strictly less than the target sequent, our intended recipient of our hypothesis would have one too many derivation steps and it could not be applied.

4.1.4 An alternative approach to substitution

Proofs about our substitution functions are difficult. Capture-avoiding renaming seems to exacerbate the issues we already face, complicating the proof by surfacing details about the new variable names.

To avoid this, we have written alternative substitution functions for de Bruijn terms that do not have to deal with renaming (as bound variables are now represented by a number, without a name). From this, the high-level proof strategy becomes as follows:

1. Observe a goal requiring showing a substitution can be derived.
2. Invoke `proves_ACONV` to transform the goal into one where we must show an alpha-equivalent derivation.
3. Rewrite the goal from an alpha equivalence relation to the equality of two de Bruijn terms.
4. Reason about the simpler de Bruijn versions of substitutions instead.

Our de Bruijn versions of the key substitution functions are written below.

Definition 12 (`db_esubst_ty_def`).

$$\begin{aligned}
\text{db_esubst_ty } \sigma \text{ (dbVar } x \text{ ty)} &= \text{dbVar } x \text{ (ty_esubst } \sigma \text{ ty)} \\
\text{db_esubst_ty } \sigma \text{ (dbBound } n) &= \text{dbBound } n \\
\text{db_esubst_ty } \sigma \text{ (dbConst } x \text{ ty)} &= \text{dbConst } x \text{ (ty_esubst } \sigma \text{ ty)} \\
\text{db_esubst_ty } \sigma \text{ (dbComb } t1 \text{ } t2) &= \text{dbComb (db_esubst_ty } \sigma \text{ } t1) \text{ (db_esubst_ty } \sigma \text{ } t2) \\
\text{db_esubst_ty } \sigma \text{ (dbAbs ty } t) &= \text{dbAbs (ty_esubst } \sigma \text{ ty) (db_esubst_ty } \sigma \text{ } t)
\end{aligned}$$

Definition 13 (`db_esubst_tm_def`).

$$\begin{aligned}
\text{db_esubst_tm } \sigma \text{ (dbVar } x \text{ ty)} &= \text{dbVar } x \text{ ty} \\
\text{db_esubst_tm } \sigma \text{ (dbBound } n) &= \text{dbBound } n \\
\text{db_esubst_tm } \sigma \text{ (dbConst } x \text{ ty)} &= \begin{cases} \text{dbConst } x \text{ ty} & \text{if FLOOKUP (SND } \sigma) x = \text{NONE} \\ \text{db } tm & \text{if FLOOKUP (SND } \sigma) x = \text{SOME } tm \end{cases} \\
\text{db_esubst_tm } \sigma \text{ (dbComb } t1 \text{ } t2) &= \text{dbComb (db_esubst_tm } \sigma \text{ } t1) \text{ (db_esubst_tm } \sigma \text{ } t2) \\
\text{db_esubst_tm } \sigma \text{ (dbAbs ty } t) &= \text{dbAbs ty (db_esubst_tm } \sigma \text{ } t)
\end{aligned}$$

Definition 14 (`db_esubst_def`).

$$\text{db_esubst } \sigma \text{ } tm = \text{db_esubst_tm } \sigma \text{ (db_esubst_ty } \sigma \text{ } tm)$$

4 Proving the Soundness of the Extension

As we can see, there is no renaming, and—compared to the regular version shown in Figure 3.3—this version is simple.

However, it is clear that we must show they are *actually* equivalent. Hence we prove one theorem for each definition to show that they truly are the de Bruijn representation of the original implementations.

Theorem 3 (db_esubst_ty_thm).

$$\frac{\begin{array}{c} \text{esubsts_ok sig } \sigma \quad \text{term_ok sig tm} \\ \text{welltyped tm} \quad \forall k \ v. \text{MEM } (v, k) \text{ env} \Rightarrow \exists n \ \text{ty}. k = \text{Var } n \ (\text{ty_esubst } \sigma \ \text{ty}) \wedge v = \text{Var } n \ \text{ty} \\ \text{esubst_ty0 env } \sigma \text{ avds tm} = \text{return subst_tm} \end{array}}{\text{db subst_tm} = \text{db_esubst_ty } \sigma \ (\text{db tm})}$$

Theorem 4 (db_esubst_tm_thm).

$$\frac{\text{only_esubsts_consts (SND } \sigma) \quad \text{term_ok sig tm}}{\text{db (esubst_tm } \sigma \ \text{tm}) = \text{db_esubst_tm } \sigma \ (\text{db tm})}$$

The proof of Theorem 3 proceeds by induction. Crucially, two auxiliary lemmas are necessary. Theorem 22 established that the free variables of a successful substitution are exactly the σ -substituted versions of the original free variables. This ensures that the de Bruijn substitution has the same effect on its variables. Theorem 5 provides the invariant that successful substitutions never introduce conflicting substitutions of bound variables. This result is critical in the abstraction case when new bindings are introduced into the environment—from this lemma we know that the fresh variables will not clash, and the induction hypothesis can apply uniformly.

Theorem 5.

$$\begin{array}{l} \forall \text{env } \sigma \text{ avds tm subst_tm.} \\ \text{term_ok sig tm} \wedge \text{esubsts_ok sig } \sigma \wedge \\ (\forall s \ s'. \text{MEM } (s, s') \text{ env} \Rightarrow \\ \exists x \ \text{ty}. s = \text{Var } x \ \text{ty} \wedge s' = \text{Var } x \ (\text{ty_esubst } \sigma \ \text{ty})) \wedge \\ \text{esubst_ty0 env } \sigma \text{ avds tm} = \text{return subst_tm} \Rightarrow \\ \forall x \ \text{ty}. \text{VFREE_IN } (\text{Var } x \ \text{ty}) \ \text{tm} \Rightarrow \\ \text{REV_ASSOCD } (\text{Var } x \ (\text{ty_esubst } \sigma \ \text{ty})) \text{ env } (\text{Var } x \ \text{ty}) = \text{Var } x \ \text{ty} \end{array}$$

Theorem 4 follows by a much simpler term induction. We note that the side condition `only_esubsts_consts` is simply a stronger version of `esubsts_ok`, that only requires showing the the substitution map σ exclusively maps constants to constants.

These theorems are important for multiple reasons. Firstly, in showing that the complex implementation of `esubst_ty0` is equivalent to the simple implementation for de

4.2 Proving the new kernel is a conservative

Bruijn terms, we seem to obtain that our complex implementation is in fact correct. Furthermore, with Theorem 3 and Theorem 4 now fully mechanised, we can more easily prove theorems about the alpha-equivalence following the application of a substitution. This is fairly simple as equal de Bruijn representations is equivalent to showing alpha equivalence.

There is also a theorem provided in the Candle sources Theorem 6 (`proves_ACONV`) that shows for anything deriveable in the logic, any alpha-equivalent term can be derived as well. This result combined with our substitution alpha-equivalence theorems give us a much cleaner path to proofs about our logic.

Theorem 6 (`proves_ACONV`).

$$\frac{\text{welltyped } c' \quad \text{ACONV } c \ c' \quad \text{hypset_ok } h' \quad \text{EVERY } (\lambda x. \text{ EXISTS } (\text{ACONV } x) \ h') \ h \quad \text{EVERY } (\lambda x. \text{ term_ok } (\text{sigof } \text{thy}) \ x \wedge x \text{ has_type Bool}) \ h'}{(\text{thy}, h') \vdash c'}$$

4.2 Proving the new kernel is a conservative

We finally arrive at the two core theorems of the project. First, Theorem 7 shows that every valid sequent in the existing kernel can be lifted to the new one—that is, everything that could be proved before can still be proved. We define lifting a theory by instantiating the new theory record with empty eliminable components.

$$\begin{aligned} \text{lift_thy}(\text{thy}) = \langle & | \text{tms} := \text{tmsof}(\text{thy}); \\ & \text{tys} := \text{tysof}(\text{thy}); \\ & \text{etms} := \text{FEMPTY}; \\ & \text{etys} := \text{FEMPTY}; \\ & \text{axs} := \text{axsof}(\text{thy}); \\ & \text{eaxs} := \emptyset \rangle \end{aligned}$$

This theorem is straight forward to prove. Performing a standard induction over the cases of the `proves` relation brings us to our conclusion quite quickly. We are required to show that every old inference rule can be emulated in the new system; since the new set of inference rules add three rules and leave the rest untouched, this is immediate.

Theorem 7 (`proves_imp_proves'`).

$$\frac{(\text{thy}, h) \vdash c}{(\text{lift_thy } \text{thy}, \{\}, h) \vdash' c}$$

In the other direction, Theorem 8 shows that everything that can be proven in the new kernel can be proven in the old kernel with a “dropped” theory representation—that is,

4 Proving the Soundness of the Extension

a theory with the eliminable and standard components merged.

$$\begin{aligned} \text{drop_thy}(\text{used_eaxes}, \text{thy}') = (&((\text{thy}'.\text{tys} \cup \text{thy}'.\text{etys}, \\ &\text{thy}'.\text{tms} \cup \text{thy}'.\text{etms}), \\ &\text{thy}'.\text{axs} \cup \text{used_eaxes}) \end{aligned}$$

This leaves us with the proof obligation that our new rules do not allow anything new to be proven—we know the existing Candle kernel only allows true statements to be proven and do not want to compromise that.

Theorem 8 (*proves' imp proves*).

$$\frac{(\text{thy}', \text{used_eaxes}, h) \vdash' c}{(\text{drop_thy } \text{used_eaxes } \text{thy}', h) \vdash c}$$

We proceed by induction over the cases of the *proves'* relation—this time, there are our three new cases that do not match up to inference rules in the existing system.

The first new case is the *elim_axioms* rule:

$$\frac{\text{theory_ok}' \text{ thy} \quad c \in \text{thy.eaxes}}{(\text{drop_thy } \{c\} \text{ thy}, []) \vdash c}$$

The “drop” function merges eliminable and standard components of the theory; hence, our goal is equivalent to showing a conclusion can be immediately derived given it is an axiom. This is clearly valid.

The second case is the *elim_discharge* rule—noting the distinction between $\vdash$ and $\vdash'$ for the existing and new kernels respectively:

$$\frac{(\text{drop_thy } \text{es1 } \text{thy}, []) \vdash c \quad (\text{thy}, \text{es1}, []) \vdash' c \quad (\text{drop_thy } \text{es2 } \text{thy}, h2) \vdash c'}{(\text{drop_thy } (\text{es2} \setminus \{c\} \cup \text{es1}) \text{ thy}, h2) \vdash c'}$$

We split on the cases of whether the conclusion c is already an axiom of the theory. If it is, then the fourth conjunct of the antecedent (*drop_thy es2 thy, h2*) matches our goal. If it is not, the result follows by the following lemma, which we prove by a standard induction over the *proves* relation:

Lemma 1 (*axioms_eliminator*).

$$\frac{(\text{thy}_2, h_2) \vdash c_2 \quad (\text{thy}_1, []) \vdash c_1 \quad \text{sigof } \text{thy}_1 = \text{sigof } \text{thy}_2}{((\text{sigof } \text{thy}_1, (\text{axsof } \text{thy}_2 \setminus \{c_1\}) \cup \text{axsof } \text{thy}_1), h_2) \vdash c_2}$$

We can now move onto the third and final goal:

$$\frac{\text{theory_ok}' \text{ thy} \quad \text{esubsts_ok } (\text{thy.ctys}, \text{thy.ctms}) \sigma \quad \text{esubsts_total } \text{thy } \sigma \quad (\text{thy}, \text{es}, h) \vdash' c \quad (\text{drop_thy } \text{es } \text{thy}, h) \vdash c}{(\text{drop_thy } (\text{IMAGE } (\text{esubst } \sigma []) \text{ es}) \text{ thy}, \text{term_image } (\text{esubst } \sigma []) h) \vdash \text{esubst } \sigma (\text{FLAT } (\text{MAP } \text{tm_names } h)) c}$$

4.2 Proving the new kernel is a conservative

This is, by far, the most difficult component of the proof, and requires many auxiliary lemmas. In words, this theorem states that any valid sequent can be substituted with some valid set of substitutions, and remain a valid sequent—that is, the substituted version could also have been proven without the new inference rules. We state and prove this in a more general form as Theorem 9.

Theorem 9 (`proves_substitutable`).

$$\frac{\text{theory_ok}(\text{sig}, \text{axs}) \quad ((\text{sig}, \text{axs}), h) \vdash c \quad \text{esubsts_ok sig } \sigma}{\left((\text{sig}, \text{IMAGE}(\text{esubst } \sigma []) \text{ axs}), \right. \\ \left. \text{term_image}(\text{esubst } \sigma []) h \right) \vdash \text{esubst } \sigma (\text{FLAT}(\text{MAP } \text{tm_names } h)) c}$$

The strategy for theorems of this form are to induct over the `proves` relation; however, in Theorem 9 this provides an insufficient inductive hypothesis. The key problem surfaces in the case of the variable instantiation inference rule: `esubst` can substitute two types into the same target type, causing free variables that are distinct before the substitution to collapse into the same one. We note that this is an issue over free variables, thus capture-avoidance is not relevant here. We instead prove the stronger statement in Theorem 10 that immediately implies Theorem 9. The stronger inductive hypothesis allows us to perform a “variable freshening” step before instantiating the abstract terms or types. This avoids the variable collapse issue.

Theorem 10 (`proves_substitutable_aux`).

$$\begin{aligned} & \forall \text{sig axs } h c. \\ & \text{theory_ok}(\text{sig}, \text{axs}) \wedge ((\text{sig}, \text{axs}), h) \vdash c \Rightarrow \\ & \quad \forall \sigma \text{ ilist}. \\ & \quad \text{esubsts_ok sig } \sigma \wedge \\ & \quad (\forall s s'. \\ & \quad \text{MEM}(s', s) \text{ ilist} \Rightarrow \\ & \quad \exists x \text{ ty}. s = \text{Var } x \text{ ty} \wedge s' \text{ has_type ty} \wedge \text{term_ok sig } s') \Rightarrow \\ & \quad ((\text{esubst_sig } \sigma \text{ sig}, \text{IMAGE}(\text{esubst } \sigma [] \circ \text{VSUBST ilist}) \text{ axs}), \\ & \quad \text{term_image}(\text{esubst } \sigma [] \circ \text{VSUBST ilist}) h) \vdash \\ & \quad \text{esubst } \sigma (\text{FLAT}(\text{MAP } \text{tm_names } h))(\text{VSUBST ilist } c) \end{aligned}$$

Theorem 10 proceeds by induction over the `proves` relation. The result is one goal for each existing inference rule. The main sources of difficulty in this proof are (a) dealing with renamed variables, and (b) showing how substitution of eliminable components commutes with other substitution mechanisms in the Candle kernel. The remainder of this section will cover the required theorems to prove this main result.

4 Proving the Soundness of the Extension

4.2.1 Safety of type substitutions

Before we can make any conclusions about the result of `esubst`, we must be sure it succeeds. Recalling the monadic definition of `esubst_ty0`, we know that under some conditions the function will return an error instead of a completed substitution. In this subsection, we state and prove Lemma 2: substitutions on well-formed terms always succeed, with an empty initial environment.

Lemma 2. *Type substitution with an initially empty environment will always return a result.*

$$\begin{aligned} & \forall \sigma \text{ tm. } \text{esubsts_ok sig } \sigma \wedge \text{term_ok sig tm} \\ & \Rightarrow \exists v. \text{esubst_ty0 } [] \sigma \text{ avds tm} = \text{return } v \end{aligned}$$

This result allows us to safely take the result of `esubst` without needing to consider failure cases. While the result seems simple, the proof was very involved—alongside the key lemmas written solely for use in this proof, it takes up just less than 300 lines of HOL4 proofs. By inspecting the definition of type substitutions in Figure 3.3, it is clear that the only source of errors are when a conflict is found in the environment association list. While abstractions do introduce new elements into the environment list, a well-formed term will not introduce conflicts. We have formalised this property, and proved the conclusion that an empty environment and a well-formed term cannot error. This is shown in Theorem 4.2.1. We note that the theorem is very long; this was required as we needed a broad induction hypothesis that covered both failure and success cases in order to reason about the full behaviour of the function.

Theorem 11. *The key `esubst_ty0` lemma*

(placeholder; the big `esubst_ty0` induction lemma)

There are further theorems about type substitution that act as both useful results, and also sanity checks to back up the claim that a returned result is truly “successful” that this theorem affords us. Theorem 12 shows that after term substitution, the term’s type is equivalent to its original type under type substitution.

Theorem 12. *A substituted term has its original type under substitution.*

$$\begin{aligned} & \text{theory_ok (sig, axs)} \wedge \text{esubsts_ok sig } \sigma \wedge \text{term_ok sig tm} \wedge \text{tm has_type ty} \\ & \Rightarrow \text{esubst } \sigma \text{ avds tm has_type ty_esubst } \sigma \text{ ty} \end{aligned}$$

4.2.2 Eliminables instantiation and type instantiation

One key case of our induction is where we must show that `INST` and `esubst` commute. `INST` is the function that the Candle kernel uses to instantiate type variables to concrete types. The commuting law is dependent on the fact that `esubst` does not affect type

4.2 Proving the new kernel is a conservative

variables. However, complexity comes from that `INST` and `esubst` both rename bound variables to avoid capture. For this reason we only claim our commuting law holds up to alpha-equivalence; the functions may assymetrically rename bound variables. We state and prove this as Theorem 13.

Theorem 13 (`esubst_INST_comm`).

$$\begin{aligned}
& \forall \text{tm}. \\
& \quad \text{theory_ok}(\text{sig}, \text{axs}) \wedge \\
& \quad \text{EVERY}(\text{type_ok}(\text{tysof}(\text{sig}))) (\text{MAP}(\text{FST})(\text{tyin})) \wedge \\
& \quad \text{esubsts_ok}(\text{sig}, \sigma) \wedge \text{term_ok}(\text{sig}, \text{tm}) \\
& \Rightarrow \text{ACONV}(\text{esubst}(\sigma, \text{avds1})(\text{INST}(\text{tyin}, \text{tm})), \\
& \quad \text{INST}(\text{MAP}(\lambda(\text{ty}, a).(\text{ty_esubst}(\sigma, \text{ty}), a))(\text{tyin}))(\text{esubst}(\sigma, \text{avds2})(\text{tm})))
\end{aligned}$$

To simplify the proof obligation, we can prove the de Bruijn version of these instead. [Abrahamsson et al. \(2022\)](#) provide a de Bruijn version of `INST`, hence we prove Theorem 14.

Theorem 14 (`db_esubst_dbINST_comm`).

$$\begin{aligned}
& \forall \text{dbtm}, \text{tyin}. \\
& \quad \text{EVERY}(\text{type_ok}(\text{tysof}(\text{sig}))) (\text{MAP}(\text{FST})(\text{tyin})) \wedge \\
& \quad \text{esubsts_ok}(\text{sig}, \sigma) \wedge \text{vtys_ok}(\text{sig}, \text{dbtm}) \\
& \Rightarrow \text{db_esubst}(\sigma, \text{dbINST}(\text{tyin}, \text{dbtm})) \\
& \quad = \text{dbINST}(\text{MAP}(\lambda(\text{ty}, a).(\text{ty_esubst}(\sigma, \text{ty}), a))(\text{tyin}))(\text{db_esubst}(\sigma, \text{dbtm}))
\end{aligned}$$

Theorem 14 follows by structural induction over the de Bruijn term `dbtm`. The only non-standard clause here is `vtys_ok`—this simply states that it only refers to types that are registered in the signature. This is a weaker version of the property of being well-typed used simply because the stronger version does not have an implementation for de Bruijn terms, and is not required.

4.2.3 Eliminables instantiation and variable instantiation

Another key case, and the most difficult we encountered is the case for free variable instantiation. This is the case of the proof that remains unfinished, and also the case whose difficulty motivated us to strengthen the induction hypothesis to the new `VSUBST` form.

(* to be completed once I have figured a little bit more out about this case. *)

4.2.4 Instantiations over abstractions

The very first case we encounter in our induction for Theorem 8 requires us to show how introducing a new abstraction commutes with instantiating eliminable components of a

4 Proving the Soundness of the Extension

sequent. Concretely, this is the case of our induction corresponding to the inference rule in Theorem 15.

Theorem 15. *If we can derive some equality in our theory, we can pick a free binder and introduce an abstraction on both sides of the equality.*

$$\frac{\forall h_i \in h. \neg(\text{VFREE_IN } (\text{Var } x \text{ } ty) \ h_i) \quad \text{type_ok } (\text{tysof } thy) \ ty \quad (thy, h) \vdash l \equiv r}{(thy, h) \vdash \text{Abs } (\text{Var } x \text{ } ty) \ l \equiv \text{Abs } (\text{Var } x \text{ } ty) \ r}$$

Noting that the inference rule is only valid when the introduced binder is free in the hypotheses, we are required to show that this inference rule still holds when under `esubst`.

$$\frac{\forall h_i \in h. \neg(\text{VFREE_IN } (\text{Var } x \text{ } ty) \ h_i) \quad \text{type_ok } (\text{tysof } sig) \ ty \quad \text{esubsts_ok } sig \ \sigma \quad (thy_\sigma, h_\sigma) \vdash \text{esubst } \sigma \text{ avds1 } (l \equiv r)}{(thy_\sigma, h_\sigma) \vdash \text{esubst } \sigma \text{ avds2 } (\text{Abs } (\text{Var } x \text{ } ty) \ l \equiv \text{Abs } (\text{Var } x \text{ } ty) \ r)}$$

This is one of the most difficult cases of our induction. From a high level, we aim to push the substitution over both sides of the equality. But we must be careful: type substitutions can rename bound variables to avoid capture, but to rely on Theorem 15 we must ensure the binder remains free in the hypotheses.

To begin the proof, we must first distribute `esubst` over both sides of the equality—this is valid as we restricted substitutions to not change the meaning of equality through `esubsts_ok`. Once this is done, we must show how `esubst` acts on abstractions to continue. Of course, we could simply unwrap all of the substitution definitions, but this would introduce a level of complexity that would perhaps not make our job much easier.

Theorem 16 (Generalised rewrite for substitutions of abstractions).

$$\begin{aligned} & \text{term_ok } sig \ (\text{Abs } (\text{Var } x \text{ } ty) \ t) \\ & \wedge \text{ theory_ok } (sig, axs) \\ & \wedge \text{ esubsts_ok } sig \ \sigma \\ & \implies \\ & \exists y \text{ body. } \text{esubst } \sigma \text{ avds } (\text{Abs } (\text{Var } x \text{ } ty) \ t) = \text{Abs } (\text{Var } y \text{ } (\text{ty_esubst } \sigma \text{ } ty)) \text{ body} \\ & \quad \wedge \text{ ACONV } body \ (\text{esubst } \sigma \text{ avds } (\text{VSUBST } [(\text{Var } y \text{ } ty, \text{Var } x \text{ } ty)] \ t)) \\ & \quad \wedge (x = y \vee \neg \text{MEM } y \text{ avds}) \end{aligned}$$

There are two main things to note here. Firstly, the `avds` list in `esubst` allows us to tell the function which names to avoid when renaming bound variables. We have proven Theorem 17 to allow us to rewrite this list for our goal as required, by Theorem 6.

The second key point for this proof is the disjunction that Theorem 16 gives us. Rewriting the goal with this gives us four cases: both binders are renamed to `y`, both stay as `x`,

only the left binder renames, only the right binder renames. We will continue to show the first of these branches, noting that all are similar in their proofs. In any branch, we have that either the binder is x —which we know is free in the hypotheses—or it is y which we force to be free in the hypotheses with a precisely-set *avds* list.

Theorem 17 (Alpha-equivalence of substitutions with different avoids lists).

$$\begin{aligned}
 & \text{esubsts_ok } \text{sig } \sigma \\
 & \wedge \text{ term_ok } \text{sig } \text{tm} \\
 & \wedge \text{ theory_ok } (\text{sig}, \text{axs}) \\
 & \implies \\
 & \text{ACONV } (\text{esubst } \sigma \text{ avds1 } \text{tm}) (\text{esubst } \sigma \text{ avds2 } \text{tm})
 \end{aligned}$$

4.2.5 Miscellaneous supporting theorems

The theorems illustrated so far do not come close to the full range of proven theorems. The full library of definitions and theorems, and their mechanised proofs, can be found at the following link:

<https://github.com/mrichards30/cakeml/tree/feature/little-theories/candle/standard/syntax>

The remainder of this subsection will follow some of these notable theorems.

Substituted theory signatures remain valid. An important lemma shows that a theory remains well-formed after a substitution. This entails every substituted axiom being well-typed, and the substituted term signature respecting the substituted types.

Theorem 18. *A well-formed theory remains well-formed after its signature and axioms are substituted.*

$$\begin{aligned}
 & \text{theory_ok}(\text{sig}, \text{axs}) \wedge \text{esubsts_ok } \text{sig } \sigma \\
 & \implies \text{theory_ok } (\text{esubst_sig } \sigma \text{ sig}) (\text{IMAGE } (\text{esubst } \sigma \text{ avds}) \text{ axs})
 \end{aligned}$$

Substituted terms remain valid. In order to prove the above theorem, the even more general result must be proven that substituted terms remain well-typed and respecting their signature. This proceeds by two key facts. The first being that a term remains well-typed after having its type substituted in `esubst_ty0`—this follows by induction over its recursive definition. The second fact is that terms remain well-typed under `esubst_tm`, provided their types have already been substituted. This distinction is important, as any term substitution have the o.

4 Proving the Soundness of the Extension

Theorem 19. *If a term is well-formed with respect to a substituted signature, then it remains well-formed after term substitution.*

$$\begin{aligned} & \forall \text{tm. theory_ok}(\text{sig}, \text{axs}) \wedge \text{esubsts_ok sig } \sigma \wedge \text{term_ok}(\text{esubst_sig } \sigma \text{ sig}) \text{ tm} \\ & \Rightarrow \text{term_ok}(\text{esubst_sig } \sigma \text{ sig}) (\text{esubst_tm } \sigma \text{ tm}) \end{aligned}$$

Theorem 20. *A term that is well-formed with respect to a signature will be well-formed when substituted under the substituted signature.*

$$\begin{aligned} & \text{theory_ok}(\text{sig}, \text{axs}) \wedge \text{term_ok sig tm} \wedge \text{esubsts_ok sig } \sigma \\ & \Rightarrow \text{term_ok}(\text{esubst_sig } \sigma \text{ sig}) (\text{esubst } \sigma \text{ avds tm}) \end{aligned}$$

Avoiding capture. We have discussed the importance of capture-avoiding substitution but have not verified the correctness of our implementation. Figure 3.3 illustrates picking new variable names using the unspecified `NVARIANT` function. Its implementation is rather uninteresting, but the result is shown in Theorem 21—that is, any new name must create a free variable in the term.

Theorem 21 (`NVARIANT_THM`). *A variable with a name picked to be a variant for the term t is truly free in t .*

$$\forall \text{ty } t \text{ n avds. } \neg \text{VFREE_IN}(\text{Var}(\text{NVARIANT } n \text{ avds } t) \text{ ty}) t$$

To show its effect in the type substitution we state and prove Theorem 22 that shows the preservation of free variables through the substitution, only changing their type.

Theorem 22 (`esubst_ty0_all_free`). *After type substitution, all free variables from the original term occur as free variables still, with their substituted type instead.*

$$\begin{aligned} & \forall \text{env } \sigma \text{ avds tm subst_tm.} \\ & \text{term_ok sig tm} \wedge \text{esubsts_ok sig } \sigma \wedge \\ & (\forall s \text{ s'}. \text{MEM}(s, s') \text{ env} \Rightarrow \\ & \quad \exists x \text{ ty. } s = \text{Var } x \text{ ty} \wedge s' = \text{Var } x (\text{ty_esubst } \sigma \text{ ty})) \wedge \\ & \text{esubst_ty0 env } \sigma \text{ avds tm} = \text{return subst_tm} \Rightarrow \\ & (\forall u \text{ uty. VFREE_IN}(\text{Var } u \text{ uty}) \text{ subst_tm} \Leftrightarrow \\ & \quad \exists \text{oty. VFREE_IN}(\text{Var } u \text{ oty}) \text{ tm} \wedge \text{uty} = \text{ty_esubst } \sigma \text{ oty}) \end{aligned}$$

4.3 Limitations

We have established that the extension to the proof-producing kernel is conservative: no new theorems can be derived that were not derivable in the base logic. This chapter has hopefully been a convincing presentation of this fact. Despite this, there are three major

limitations with the solution presented over chapters 3 and 4: the first being social (whether users will find the system convenient), then the source of error that comes with hand-translating the specification onto a standard non-verified ITP like HOL4, and finally, the limitations of the substitution functions.

4.3.1 The usefulness of the system

We have verified the conservativeness of the extension to the proof-producing kernel, but it is clearly not enough to guarantee the correctness of the system in a broader sense. The aim of a module system like the one presented here remains to be *to make theorem proving more convenient*—of course, this is one of public opinion rather than anything provable. We have discussed examples such as the worked example of interpreting theorems over monoids as theorems over natural numbers, but without much more user testing, it is unclear whether the limitations on when the modules can be instantiated might be too limited as to achieve further useful results.

4.3.2 Implementing the specification

The verified system is implemented on Candle, but the aim of the project is to create a specification that can be lifted onto HOL4. Candle uses code extraction from HOL4 to obtain a verified binary—HOL4 does not. To port the system to HOL4, the code will be inspected and re-written manually into ML. While this is not a particularly difficult process, as HOL4’s higher-order logic closely mirrors ML’s own syntax, it is clear that this may be a large source of error and deviation from the verified specification. This concern is not only hypothetical: [Abrahamsson et al. \(2025\)](#) report that a brief soundness bug was introduced through a mistake while hand-translating the code. This bug was caught and fixed quickly enough, but highlights the risks of the method.

4.3.3 Limitations of the substitution functions

As shown in the report, the type substitution functions deliberately only support substituting monomorphic types. For example, an abstract type such as `monoid` can be replaced with `int` after showing it fits the former’s specification. This has two main consequences:

- **Simplicity:** The type substitution function is simpler to implement and reason about, making it easier to verify. It avoids confronting issues raised by allowing the function to introduce new type variables, or substituting higher-kinded types.
- **Loss of expressiveness:** In practice, this frustrate users. It is easy to imagine one wanting to substitute a module about `monoid` on the polymorphic list α , but this is impossible as the substitutions are not allowed to introduce new type variables under our implementation.

These limitations will be addressed in the following chapter.

Related Work

Module systems in theorem provers are not a novel idea; previous work has been done in this area that both contextualises and motivates ours. Other mainstream theorem provers, such as Isabelle and Rocq, have these features (and have had for many years). We will look at these and compare them to our project. There is also more theoretical research on module systems (theory interpretations) that we can discuss.

Little Theories. [Farmer et al. \(1992\)](#) describe a theoretical framework for a logic depicting what we referred to as a module system. This logic was designed to model “little theories” (what we might have called modules) as first class constructs in the logic. This allows modular reasoning, with the benefit of being able to reason about the modules themselves; the paper argues this ability is beneficial. [Farmer et al. \(1993\)](#) implements a first version of this. Recent developments on this topic develop a new logic called Alonzo ([Farmer and Zvigelsky, 2025](#)); however, there is no proof assistant yet that implements this logic. The difference between these and the intended implementation for Candle is that we are adding a module system onto to an existing, adopted logic, rather than building a new logic from the ground-up to support the feature. We have the added task of retaining backwards compatibility while making the system convenient to use.

Isabelle Locales. [Kammüller et al. \(1999\)](#) present the first iteration of Isabelle’s module system, called *locales*. Motivated by abstract algebra as the prime example, locales allow proofs to be written on abstract structures then transported onto concrete types and terms, by showing the target constants/types obey the assumptions of the module. However, the antiquity of the system justified a rewrite and extension later. [Ballarin \(2006\)](#) presents a modernised version of locales, including a new syntax to optimise ease of use. The system presented here does not touch Isabelle’s logic. It is essentially a macro system and front-end over the logic. This results in a trade-off between simplic-

5 Related Work

ity and expressiveness. The “proof” of conservativeness for Isabelle’s locales system is therefore trivial, as the logic is not affected. However, polymorphic constants cannot be reasoned about in within modules; this limitation differentiates the approach from ours. In our system, modules are first-class theorems that contain eliminable types and terms, hence there are no restrictions on what can be used in local reasoning, other than what would stop the module from being usefully instantiated. Hence we posit our system is more expressive than Isabelle’s locales. Conversely, Isabelle’s locales do not face the restrictions our contribution has: type variables and non-nullary types are not disallowed. However, it seems that this could be feasibly remediated. This is discussed in Chapter 6.

Rocq Modules. In the limitations of their paper, Ballarin (2006) suggest that module systems are more naturally encoded in type theory. This is reflected in the module system in Rocq (formerly Coq). Unlike Isabelle’s implementation, Rocq’s module system constitutes an extensions of the system’s calculus, and thus faces a similar need to justify correctness as this thesis does. While the clear difference between these systems and ours lie in that they are based on dependent type theory. Chrzaszcz (2004) provides a proof for the correctness of the Rocq module system: that the extension to the calculus is conservative. Published in 2003, this proof is before the time of verified (and extracted) interactive provers in the vein of Candle and thus exists on a theoretical model of the prover. Sozeau et al. (2020) discuss the *MetaCoq* project in which a model of Rocq is formalised within Rocq itself—the aim of the project is to verify the full system. As of today, MetaCoq does not include a formalisation of the module system. Sozeau (2021) suggest it may remain excluded and a non-mechanised proof may be considered instead; but whichever direction it follows, consideration of verifying their module system is ultimately left to future work.

Relation to our Contribution. Finally, we note that while other interactive theorem provers have solutions for a theory-level module system, this thesis presents the implementation for a verified theorem prover, with an underway effort to verify the extension. Other theorem provers such as Rocq may have ongoing verification efforts in MetaCoq, but the relative simplicity of HOL affords us a completely verified system already in Candle. As far as the author is aware, this presents us with the opportunity to work on most comprehensive verification of the correctness of this kind of module system today. However, other theorem provers do not face the restrictions on type instantiation that we have introduced in ours. There is a sizable amount of future work that must be done to bring our system up to the level of the others; pathways to address this are described Chapter 6.

Discussion and Future Work

6.1 Continuing the proof

the remaining work. to be completed once I have tried the VSUBST case a bit more and know exactly what is left.

6.2 Porting the system to HOL4

Currently, the specification only exists in the logic of Candle; the aim of the project is to eventually bring the module system into HOL4. This entails requiring a parser to be written for modules, and an intuitive syntax for opening instantiated modules into a theory. As modules systems are well-known features in other theorem provers, it seems that the syntax for handling these is essentially a solved problem. The syntax could reasonably mirror Isabelle’s locales.

Furthermore, the theory structure needs to be extended to demarcate the set of eliminable types and terms from the standard ones. In this project this was done by changing the theory tuple to the extended theory record. In the HOL4 implementation, however, it seems like it may be simpler to keep a “database” structure that maps theorems to their eliminable components. This is easier because HOL4 is written in SML and therefore gives us access to mutable data structures and references. It seems that this storage method would make the implementation process simpler, while not considerably changing the abstraction that we have written our theorems over.

6.3 Permitting polymorphic substitutions

The specification forms a “version 1” of the module system. While these restrictions make the system simpler to reason about, they ultimately provide a less expressive

system in general. Obvious future work for the project thus involves weakening these restrictions. We can recall that the main usability limitation that users may encounter was that the system did not handle polymorphic types. As discussed in Chapter 5, Alonzo similarly only allows nullary monomorphic type substitution; however, the key difference is that Alonzo’s type system does not support type polymorphism in general, while HOL does.

We have discussed how allowing our system to introduce type variables would be desirable (e.g., to instantiate `monoid` as `list  $\alpha$`). This was not explored mainly due to time constraints, given the complexity that such a system would entail. The main problem that must be solved is how to handle type variables conflicts—that is, we cannot naively apply the substitution that maps `monoid` to `list  $\alpha$` , as α may already be constrained in the target theorem’s context. Instead, it seems we must add new wiring through `esubst` to mark which type variables must be avoided; this could perhaps be done in a similar style to the `avds` list of bound variable names we have already implemented.

This certainly seems feasible, but it might be wise to wait until this first version of the system is proven correct before introducing further complexity.

6.4 A direct proof of soundness

The verification effort presented in this thesis focuses on the specification introduced in Chapter 3. Our main result aims to establish that the new inference rules are conservative over the existing Candle inference system. Formally, we aim to prove that for any sequent $(thy, h) \vdash c$ derivable in the original system, the substituted sequent is also derivable.

This conservativity theorem implies that we are not allowing anything to be derived that could not already be derived without the extension. Because the soundness of Candle has already been proven by Abrahamsson et al. (2022), it is sufficient to show that the set of everything derivable has stayed the same; this directly implies our system has remained sound.

Beyond Conservativity. Showing conservativity was not the only option to prove the system correct. Chapter 4 only discussed proofs about the syntax of the system, and not the semantics. We could have instead proven that the substitution mechanism we define in Chapter 3 can never transform a sound sequent into an unsound one. As all required proofs have not yet been fully mechanised, it is worth considering the possibility that the extension might not actually be conservative—though it could still be sound. That is, the extended system might admit derivations that were not previously expressible in Candle but are nevertheless semantically valid. Proving such a result would require a direct soundness proof for the extended system, rather than the argument for conservativity that we presented. We also note that even if the extension is conservative as we expect, a soundness argument would have still been a valid method to prove its correctness.

Although this thesis does not pursue this line of reasoning, it remains conceptually

interesting. Intuitively, our inference rules merely allow a “shortcut”: we require a proof over the abstract types and terms, then argue that the proof is therefore valid over concrete types and terms under the same axioms. Since these instantiated modules eventually require a regular proof that is sound without deferring reasoning to a module—thus using proofs that were proved sound in the existing model of Candle—it is difficult to imagine that new, unsound conclusions could be produced. Thus, we have reason to believe that the system is both conservative and sound.

Relation to Existing Work. The Candle soundness proofs presented by [Abrahams-son et al. \(2022\)](#) already contains a library of supporting lemmas, such as results for term and type instantiation rules like `INST` and `VSUBST`. It seems that to study and prove results about the semantics of `esubst`, we could build our work of these. However, there are some key differences:

1. `esubst` handles monomorphic, nullary types and constants, rather than polymorphic type variables (like `INST`) or instantiating free variables to concrete terms of the same type (like `VSUBST`).
2. Our rules substitute axioms, which is not handled by either of the existing substitution rules.

In order to study the semantics of `esubst` we would have to properly research what these differences require in terms of new proof obligations. While this direction seems to be a valuable contribution, time constraints limited our focus to the conservativity approach.

Conclusions

In this thesis, we have described an extension to the logic of HOL-based theorem provers to support a module system that enables theorem reuse. We have also explored proving correctness of the proposed design. The implementation has been integrated into the Candle source code written in the HOL4 theorem prover, and the correctness theorems written in terms of the existing verified sources of the Candle project.

Many limitations of the project exist; however, it seems feasible for future work to extend this to an extent under which the contribution could become a useful feature for everyday proving in HOL-based ITPs. We have presented some key limitations in Section 4.3, and discussed ways to remedy them in Chapter 6.

As we have stated before, modules in theorem proving are certainly not a novel concept. In the late 1990s to early 2010s, mainstream theorem provers tackled their solutions to theorem proving module systems. However, it was only in the mid 2010s that the comprehensive verification of theorem provers became of particular interest. We present our contribution in a modern-day context, where it is more feasible than ever to make more guarantees during theorem prover development. We have now described and justified an implementation specification for a new module system for the HOL-family of ITPs, and begun a mechanised verification unlike what has been done before. It remains incomplete, but we have discussed how the work could be continued in the future from our contribution of a large library of theorems written about the system.

Bibliography

- ABRAHAMSSON, O.; MYREEN, M. O.; KUMAR, R.; AND SEWELL, T., 2022. Candle: A verified implementation of hol light. *13th International Conference on Interactive Theorem Proving (ITP 2022)*, (2022). [Cited on pages 3, 8, 22, 31, 40, and 41.]
- ABRAHAMSSON, O.; MYREEN, M. O.; NORRISH, M.; KANABAR, H.; AND POHJOLA, J. Å., 2025. Fast, verified computation for hol itps. *Journal of Automated Reasoning*, 69, 1 (2025), 7. [Cited on pages 8 and 35.]
- BALLARIN, C., 2006. Interpretation of locales in isabelle: Theories and proof contexts. In *Mathematical Knowledge Management*, 31–43. Springer Berlin Heidelberg, Berlin, Heidelberg. [Cited on pages 37 and 38.]
- CHRZASZCZ, J., 2004. Modules in coq are and will be correct. In *Types for Proofs and Programs*, 130–146. Springer Berlin Heidelberg, Berlin, Heidelberg. [Cited on page 38.]
- ENDERTON, H. B., 1972. *A Mathematical Introduction to Logic*. Academic Press, New York,. [Cited on pages 2 and 7.]
- FARMER, W. M.; GUTTMAN, J. D.; AND JAVIER THAYER, F., 1992. Little theories. In *International Conference on Automated Deduction*, 567–581. Springer. [Cited on page 37.]
- FARMER, W. M.; GUTTMAN, J. D.; AND THAYER, F. J., 1993. Imps: An interactive mathematical proof system. *Journal of Automated Reasoning*, 11, 2 (1993), 213–248. doi:10.1007/BF00881906. <https://doi.org/10.1007/BF00881906>. [Cited on page 37.]
- FARMER, W. M. AND ZVIGELSKY, D. Y., 2025. Monoid theory in alonzo: A little theories formalization in simple type theory. <https://arxiv.org/abs/2312.05658>. [Cited on page 37.]
- GORDON, M.; MILNER, R.; MORRIS, L.; NEWAY, M.; AND WADSWORTH, C., 1978. A metalanguage for interactive proof in lcf. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’78 (Tucson,

Bibliography

- Arizona, 1978), 119–130. Association for Computing Machinery, New York, NY, USA. doi:10.1145/512760.512773. <https://doi.org/10.1145/512760.512773>. [Cited on page 7.]
- KAMMÜLLER, F.; WENZEL, M.; AND PAULSON, L. C., 1999. Locales a sectioning concept for isabelle. In *Theorem Proving in Higher Order Logics*, 149–165. Springer Berlin Heidelberg, Berlin, Heidelberg. [Cited on page 37.]
- MYREEN, M. O. AND OWENS, S., 2012. Proof-producing synthesis of ml from higher-order logic. *SIGPLAN Not.*, 47, 9 (Sep. 2012), 115–126. doi:10.1145/2398856.2364545. <https://doi.org/10.1145/2398856.2364545>. [Cited on page 8.]
- PIERCE, B. C., 2002. *Types and Programming Languages*. The MIT Press, 1st edn. ISBN 0262162091. [Cited on page 6.]
- SLIND, K. AND NORRISH, M., 2008. A brief overview of hol4. In *Theorem Proving in Higher Order Logics*, 28–32. Springer Berlin Heidelberg, Berlin, Heidelberg. [Cited on pages 7 and 8.]
- SOZEAU, M., 2021. Touring the metacoq project (invited paper). *Electronic Proceedings in Theoretical Computer Science*, 337 (Jul. 2021), 13–29. doi:10.4204/eptcs.337.2. <http://dx.doi.org/10.4204/EPTCS.337.2>. [Cited on page 38.]
- SOZEAU, M.; ANAND, A.; BOULIER, S.; COHEN, C.; FORSTER, Y.; KUNZE, F.; MALECHA, G.; TABAREAU, N.; AND WINTERHALTER, T., 2020. The metacoq project. *Journal of Automated Reasoning*, 64, 5 (2020), 947–999. doi:10.1007/s10817-019-09540-0. <https://doi.org/10.1007/s10817-019-09540-0>. [Cited on page 38.]
- TROELSTRA, A. S., 1996. *Basic Proof Theory*. Cambridge University Press, New York. [Cited on pages 23 and 24.]